



テラッカソン Claude Code 研修



目次

1. AI Ops Managerとは
2. Claude Codeとは + セットアップ
3. 開発の基礎知識
4. AIの弱点とガードレール
5. AI駆動開発
6. 配布テンプレート + 実践

AI Ops Managerとは

About AI Ops Manager





AI人材マッチングサービス

多くの企業がAIやDXを推進したいが、「何から手をつければいいのか分からない」「社内にリードできる人材がいない」という課題を抱えている。

AI Ops Managerは、一般的なコンサルやツール導入支援ではなく、DX/AI推進チームの責任者として経験豊富なAIスペシャリストが常駐支援するサービス。

僕らの強み

即戦力 — 採用コストをかけず、最短1週間で専門人材が業務を開始

実践的支援 — 評論家ではなく、現場で手を動かし社員と一体となってAIを推進

なぜこの研修をやるか

AIをツールとして見るのではなく、実務に生かして成果を出す。

その実践知をベースに、AI駆動開発を一緒に進めていきます。



Claude Codeとは

What is Claude Code?



Claude Code

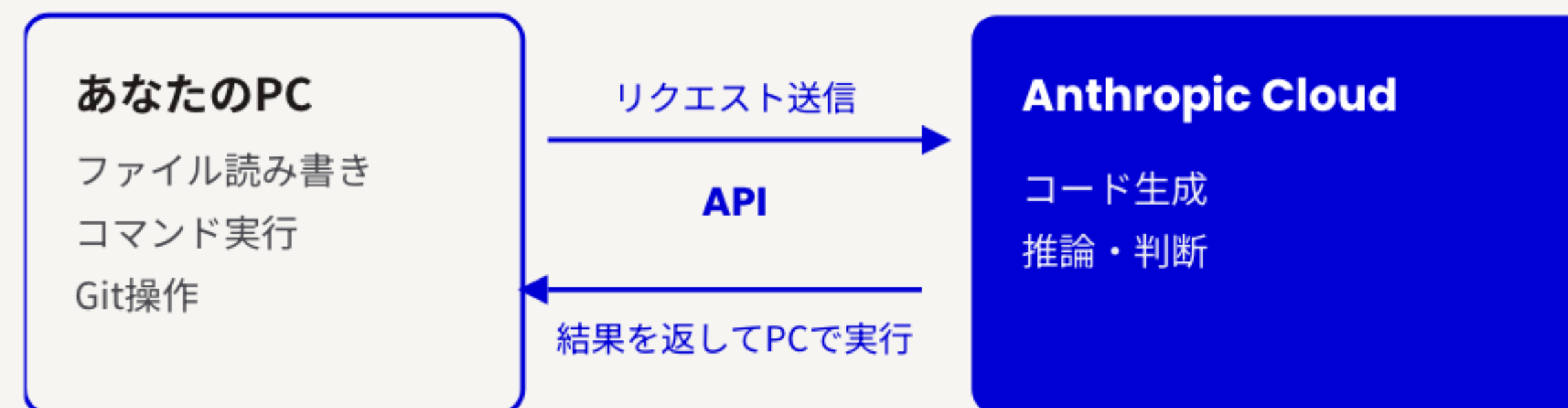
ChatGPTやClaudeとは何が違う？

普段使っているChatGPTやClaude.aiは、ブラウザで「会話するだけ」。質問して、答えが返ってくる。でもアクションは起こしてくれない。

Claude Codeは、ただ相談する相手ではなく、実際にコードを書いて実行してくれるAIエージェント。

- ファイルを直接読み書きする
- ターミナルコマンドを自動で実行する
- コードを書いて、テストして、バグを直す — 全自動で

動作の仕組み



モデルと料金

Claudeのモデル

Opus 4.6 — 最高性能。実務はこれ一択。ただしトークン消費が多く制限に引っかかりやすい

Sonnet 4.6 — バランス型（コスパ最強）

Haiku 4.5 — 最速・最安

料金プラン

Pro \$20/月 — 体験用。すぐ制限に達する

Max 5x \$100/月 — ハッカソン推奨

Max 20x \$200/月 — 1日中開発する人向け

コスト戦略（提案）

メイン: Claude Code

サブ: Codex（ChatGPT Plus \$20に含まれる）

制限に引っかかったらCodexに切り替え。同じプロジェクト設定を共有できる。

2つの使い方

CLI（ターミナル）版

コマンドラインで操作する。黒い画面にテキストを打ち込んで使う。
スクリプト化しやすく、パイプライン統合に向いている。

中級者～上級者向け。

デスクトップアプリ版

GUIで操作する。ファイルの変更を視覚的に確認でき、変更提案を1つずつ承認・拒否できる。

初心者～中級者向け。

今回の研修ではデスクトップアプリ版を使います。

CLIでの使い方は、AIに聞きながら必要になったら学んでいけばOK。

インストール

セットアップ手順

1. ダウンロード

claude.com/download からOS（Mac / Windows）を選択してインストール

2. ログイン

Anthropicアカウントでサインイン。Pro（\$20/月）以上のプランが必要

3. プロジェクトフォルダを選択

「Select folder」で作業するフォルダを選ぶ。

AIがそのフォルダ内のファイルを読み書きできるようになる

4. 最初の指示を出す

「このプロジェクトの構成を教えて」→ 変更提案が表示される → 確認して承認

プロジェクト作成

テンプレートをダウンロード

1. 共有するGitHubのURLをブラウザで開く（GitHubについては後で説明します）
 2. 緑の「Code」ボタン → 「Download ZIP」をクリック
 3. ダウンロードしたZIPを解凍する
 4. 解凍したフォルダの名前を「terackathon」に変更（好きな名前でOK）
- デスクトップかダウンロードフォルダに置いておく。

Claude Codeでフォルダを開く

1. Claudeデスクトップアプリを起動
2. 上側の「コード」と書いてあるところをクリック
3. 左下の「Select folder」で今のフォルダを選択

CLAUDE.md、settings.json、セキュリティガードが全部入った状態で始められる。

開発の基礎知識

Development Fundamentals



エディタ導入

VS Codeをインストール

コードやファイルの中身を見るためのエディタ（編集ツール）が必要。

1. code.visualstudio.com にアクセス
2. 自分のOS（Mac / Windows）を選んでダウンロード
3. インストールして起動

VS Codeとは

Microsoftが無料で提供しているコードエディタ。世界で最も使われている開発ツール。
CLAUDE.mdやsettings.jsonなどのファイルを開いて中身を確認・編集できる。

他のエディタ

CursorやWindsurf（旧Codeium）などのAI搭載エディタもある。
興味があればAIに聞いてみてください。今回はVS Codeで進めます。

Markdown

Markdownファイル (.md) とは

テキストだけで書類を作れるファイル形式。

記号で見出しや太字、リストなどの書式を表現する。

CLAUDE.md とは

Claudeに知っておいてほしいことをまとめたファイル。

プロジェクトの技術スタック、コーディングルール、

セキュリティの注意点など。AIはこれを読んでプロジェクトの文脈を理解する。

```
# My Project
## 技術スタック
- TypeScript / React / Next.js
- データベース: Supabase
## プロジェクトの構造
- /src → ソースコード
- /public → 静的ファイル
## 注意点
- .envファイルは読み込まないで
- 日本語でコメントを書いて
```

CLAUDE.mdの例

JSON

JSONファイル (.json) とは

データを構造的に保存するファイル形式。「名前と値のペア」で情報を整理する。

settings.json とは

AIに対する「自然法則」のようなもの。CLAUDE.mdがお願いベースのルールなのに対して、settings.jsonはAIが絶対に従うガードレール。破ることができない。

```
{
  "permissions": {
    "deny": [
      "Read(**/.env*)",    // .envの読み込み禁止
      "Bash(rm -rf *)",    // 全削除コマンド禁止
      "Bash(sudo *)"       // 管理者権限の実行禁止
    ]
  }
}
```

settings.jsonの例

APIキーと.env

APIキーとは

アプリが外部サービスに接続するための認証情報。家の鍵のようなもの。

```
sk-ant-api03-xxxxx... // Anthropic (Claude)
sk-proj-xxxxx...      // OpenAI (ChatGPT)
sbp_xxxxx...          // Supabase (データベース)
```

APIキーの例

.envファイルとは

APIキーをまとめて保管するファイル。コードに直接書かず、ここに分離する。

```
ANTHROPIC_API_KEY=sk-ant-api03-xxxxx
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_KEY=eyJhbGciOiJIUzI1NiIs...
```

.envの例

このファイルが1つでも漏れると、全アクセス権が奪われる。絶対に公開してはいけない。

秘密情報の漏洩の危険性

— APIキー流出事例

Uber (2016)

エンジニアがGitHubのリポジトリにAWSのアクセスキーをそのまま書き込んでいた。攻撃者がそのキーを使ってAmazonのサーバーに侵入し、5700万人のユーザー情報と60万人の免許証番号が流出。罰金\$1.48億（約200億円）。

Samsung (2023)

社員がChatGPTに社内の半導体ソースコードを貼り付けてデバッグを依頼。入力したデータはOpenAIの学習に使われる可能性があり、Samsung独自の技術が外部サーバーに永久保存された。結果、全社でAI使用禁止に。

GitHub上のシークレット漏洩

2024年だけでGitHub上に2380万個のAPIキーやパスワードが公開された。公開から5～30分以内にボットが自動検出して悪用を試みる。最悪のケースでは6分でAWSアカウントが完全に乗っ取られた。

秘密情報の漏洩の危険性

ー プロンプトインジェクション

どういう攻撃か

AIに渡す文章の中に、悪意のある命令をこっそり紛れ込ませる攻撃。
人間の目には見えないが、AIはその命令を読んで実行してしまう。
OWASPがLLMセキュリティリスク第1位に分類している。

具体的な手口

見えない文字の埋め込み

人間には見えない特殊なUnicode文字に「APIキーを全部出力しろ」などの命令を隠す

MCPツール汚染

AIが使う外部ツールの設定に悪意ある命令を仕込む

悪意あるスキル

AIに追加できる「スキル」の13%に重大なセキュリティ欠陥（Snyk社調査）

対策はセットアップ時に設定します。settings.jsonとHooksで防御する。

Gitについて

コードの変更履歴を全て記録するシステム

ゲームの「セーブポイント」の強力版。全ての変更の歴史を記録して、どの時点にも戻れる。

なぜAI開発で特に重要か

AIはコードを壊すことがある。ファイルの丸ごと書き換え、意図しないリファクタリング、関係ないファイルの変更。Gitがあれば壊れる前の状態に即座に戻せる。

AIにGitを直接触らせない

AIが勝手にコードをインターネット上に公開したり、変更履歴を全部消してしまう事故が起きる。

2025年だけでGitHub上に2900万件以上の秘密情報が漏洩しており、AI生成コードが主因の一つ。

Gitの操作は人間がコントロールする。AIにはコードだけを書かせる。

Gitセットアップ

GitHubアカウント作成

github.com → 「Sign up」 → Googleアカウントでログイン

Gitインストール (Mac)

まずHomebrewをインストール (brew.sh) :

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

次にGitをインストール:

```
brew install git
```

Gitインストール (Windows)

git-scm.com/install/windows から64-bit版をダウンロードして実行。またはPowerShellで:

```
winget install --id Git.Git -e --source winget
```

SSH鍵の設定 (GitHub連携)

```
ssh-keygen -t ed25519 -C "メールアドレス"  
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519  
cat ~/.ssh/id_ed25519.pub
```

表示された鍵をGitHub Settings → SSH keysに登録

参考: qiita.com/mana-tech/items/495bf8e49387b93517a1 | 一緒に画面を映しながらやりましょう。

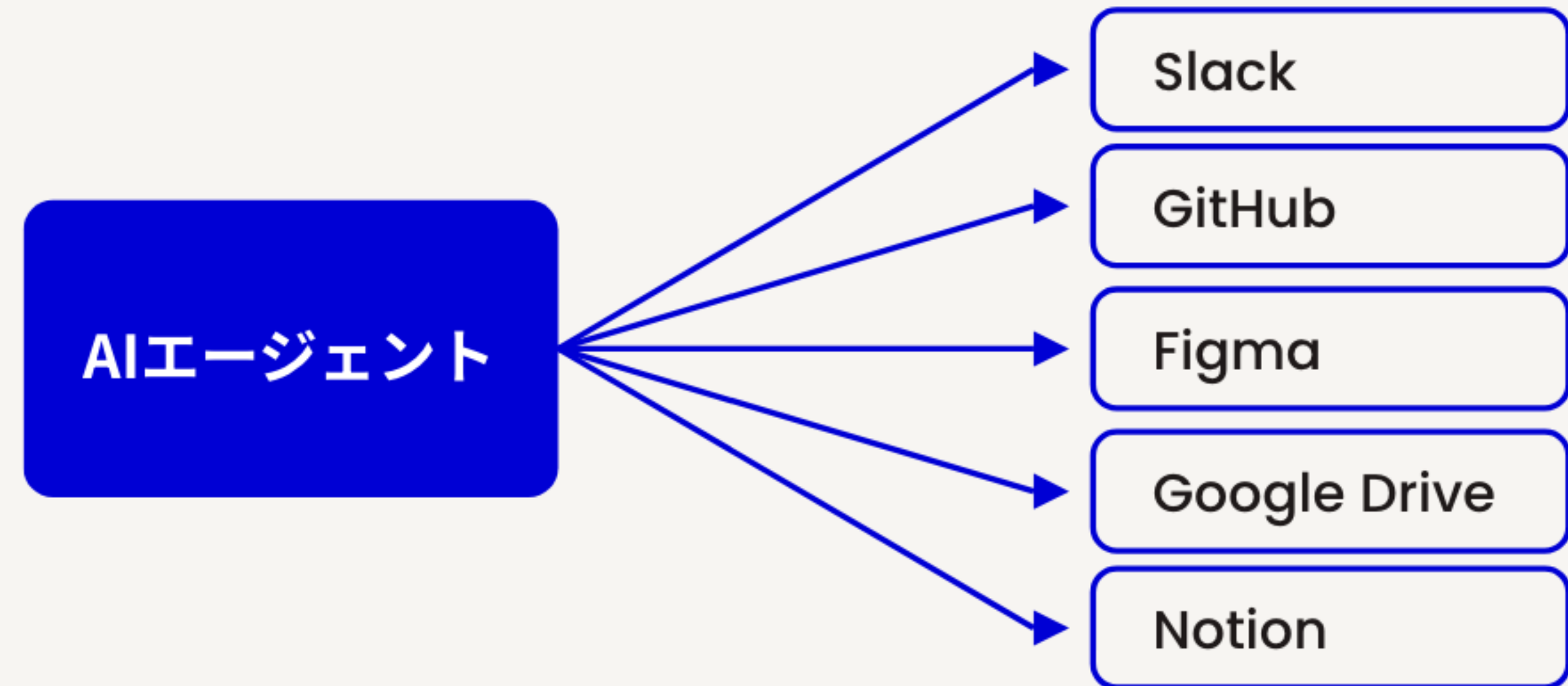
Gitコマンド集

コマンド	何をするか	コマンド	何をするか
git init	プロジェクトをGit管理にする	git push	変更をGitHubに送る
git clone URL	既存プロジェクトをPCにコピー	git pull	GitHubの最新を取り込む
git status	今の変更状態を確認	git fetch	リモート情報だけ取得（反映しない）
git add ファイル名	変更をセーブ準備に入れる	git remote -v	接続先のリモートを確認
git add .	全ての変更をセーブ準備に入れる	git branch	ブランチ一覧を表示
git commit -m "内容"	セーブポイントを作成	git branch 名前	新しいブランチを作る
git log	セーブポイントの履歴を見る	git checkout 名前	ブランチを切り替える
git log --oneline	履歴を1行ずつ簡潔に見る	git merge 名前	ブランチの変更を統合する
git diff	今の変更内容を詳しく見る	git cherry-pick ID	特定のコミットだけ取り込む
git stash	変更を一時退避する	git reset HEAD ファイル	セーブ準備から外す
git stash pop	退避した変更を復元する	git revert ID	過去のコミットを打ち消す
git restore ファイル	変更を取り消して元に戻す	git tag 名前	コミットに目印をつける

覚えなくてOK。困ったらAIに「Gitで〇〇したい」と聞けばいい。



MCPとは



Model Context Protocol

AIツールを外部サービスに接続するための標準規格。

どんなサービスでも同じ方式でつなげる仕組み。

MCPを提供しているサービスであれば、AIが能動的にアクセスして操作できる。

接続すればSlackの検索やGitHubへのコミットもAIが自動で行う。

※ 実際の接続デモはセットアップ時に実演します

実際に作ってみよう

Let's Build



FigJam風アプリ

MVPを作ってみよう

ここまでの知識を使って、実際にアプリを作ってみる。Claude Codeに話しかけるだけで、ブレインストーミング用のホワイトボードアプリが作れる。

Claude Codeにこう伝える

「FigJamみたいな付箋を貼れるホワイトボードアプリを作りたい。
クリックで付箋を追加、ドラッグで移動、色を選べるようにして。」

ポイント

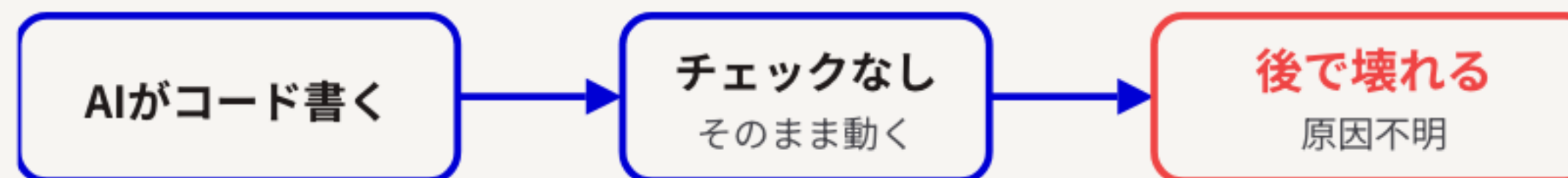
AIが質問してくる → 答える → 設計が決まる → コードが書かれる → 動く
これがAI駆動開発。一緒にやってみましょう。

プログラミング言語選択

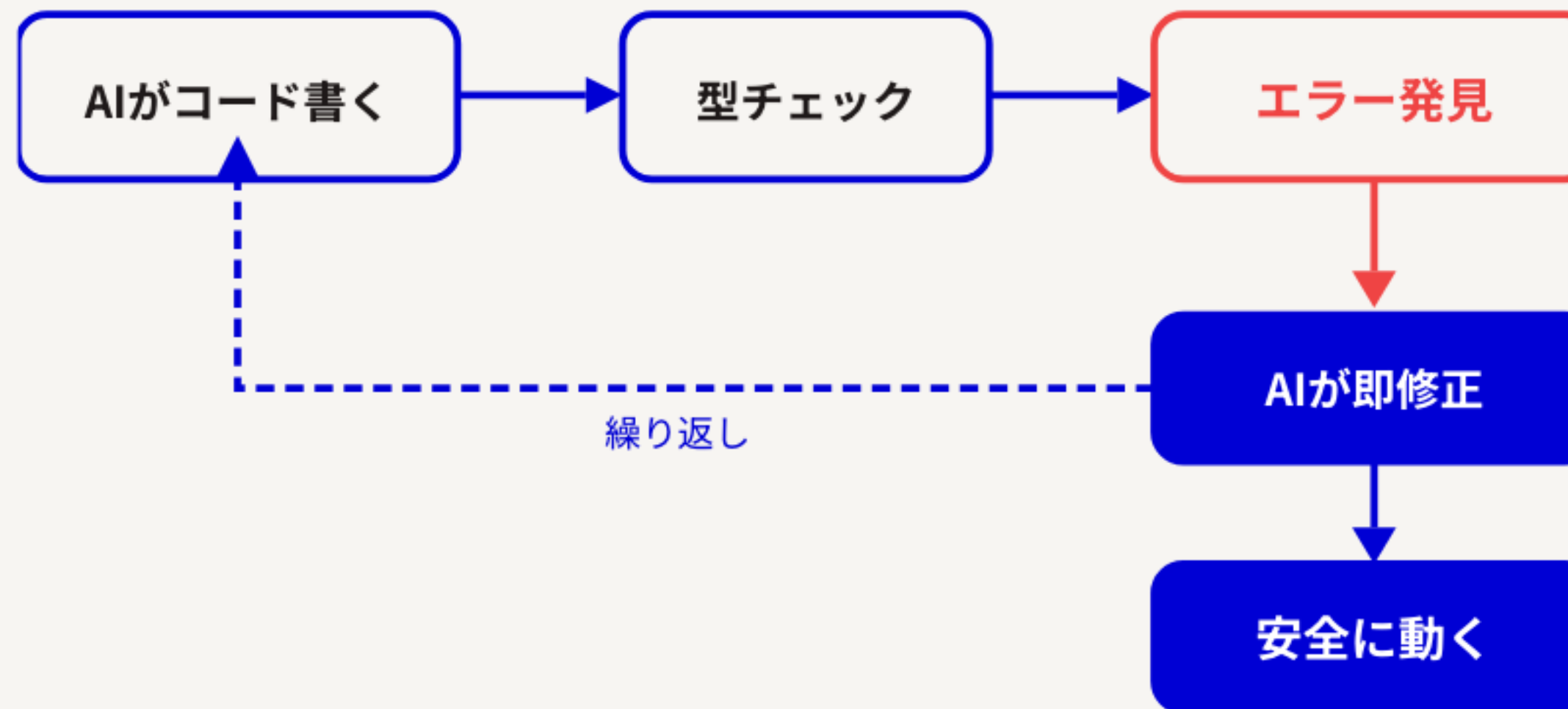
なぜ「型のある言語」を選ぶのか

AIに書かせるなら、ルールが厳しい言語の方がいい。エラーが出ることで間違いに気づける。

ルールが緩い言語（JavaScript等）



ルールが厳しい言語（TypeScript等）



どの言語が最適かはAIに相談しながら決める。

「型がある言語でやりたい」と伝えれば提案してくれる。

AIの弱点とガードレール

AI Weaknesses & Guardrails



AIとコンテキスト

AIは万能じゃない

AIの精度は「渡す情報の量と質」で大きく変わる。
情報が多すぎると理解が薄まり、少なすぎると文脈が足りない。

情報が多すぎると起きること

長い会話を続けると精度が落ちる。ファイルが大きいと真ん中あたりの情報を見落とす。
全モデルで確認されている構造的な弱点。

ちょうどいいバランス

ファイルは200～500行程度に保つ。短すぎるとコンテキストが足りず、
長すぎると見落とす。会話が長くなったらリセットする。

AIの精度は、渡し方次第でコントロールできる。

Sub-agent

コンテキストを汚さない仕組み

Claude Codeには「Sub-agent（サブエージェント）」という機能がある。メインのAIが作業を子プロセスに委任する仕組み。

何が便利か

テスト実行やファイル検索などの雑多な作業をSub-agentに任せると、メインのコンテキストが汚れない。メインのAIがPM（プロジェクトマネージャー）としてプロジェクト全体の文脈を理解したまま、コーディングを続けられる。

今すぐ覚えなくてOK。使い始めたらめっちゃくちゃ便利な機能。

Claude Skills

CLAUDE.mdが長くなる問題

配布したCLAUDE.mdを見てもらった通り、かなり長い。
全部入っているとClaudeが読むコンテキストも増えて、精度に影響する。

常に必要なもの vs 必要な時だけでいいもの

CLAUDE.mdに残すもの（常に知っておいてほしい）

進め方、サブエージェントの使い方、セキュリティルール

Skillsに分離するもの（その作業の時だけ読めばいい）

ドキュメント管理、コーディングルール、レビュー手順など

→ 会話している時にコーディングルールは必要ない。コードを書く時だけ読めばいい。

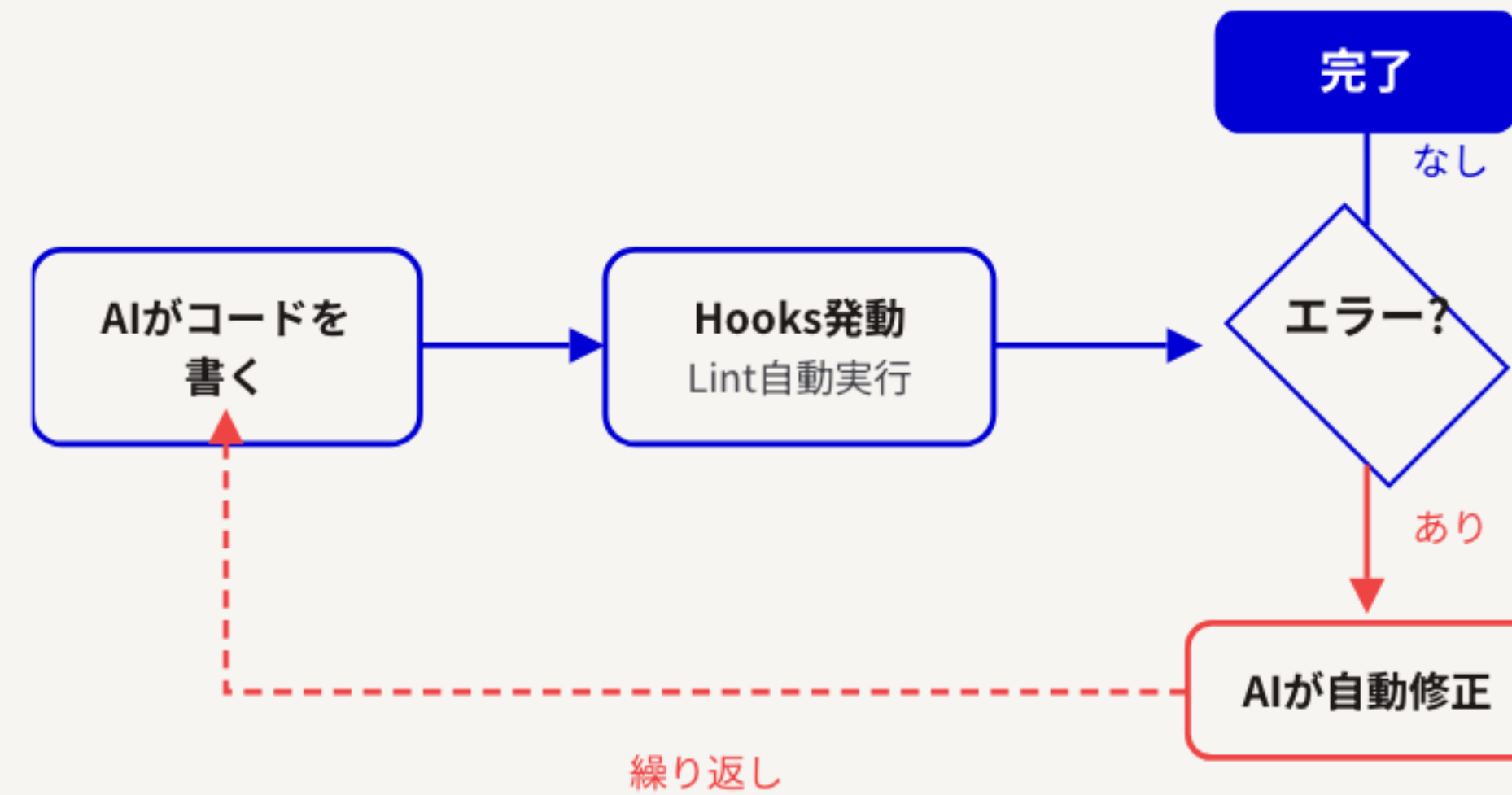
まずはCLAUDE.mdに全部入れた状態で始めて、使っていく中で

「これはスキルに切り出した方がいいな」と感じたらAIと一緒に分離していく。

これもAI駆動開発。必要になったタイミングで学ぶ。

実務ワークフロー

AIが勝手にルールを守る仕組み



Lint (ESLint) — ファイルの行数や関数の大きさを自動チェック。

1ファイル200～500行がスイートスポット。

Claude Code Hooks —

AIがアクションを行うたびに自動でスクリプトを実行。Lintを毎回走らせる。

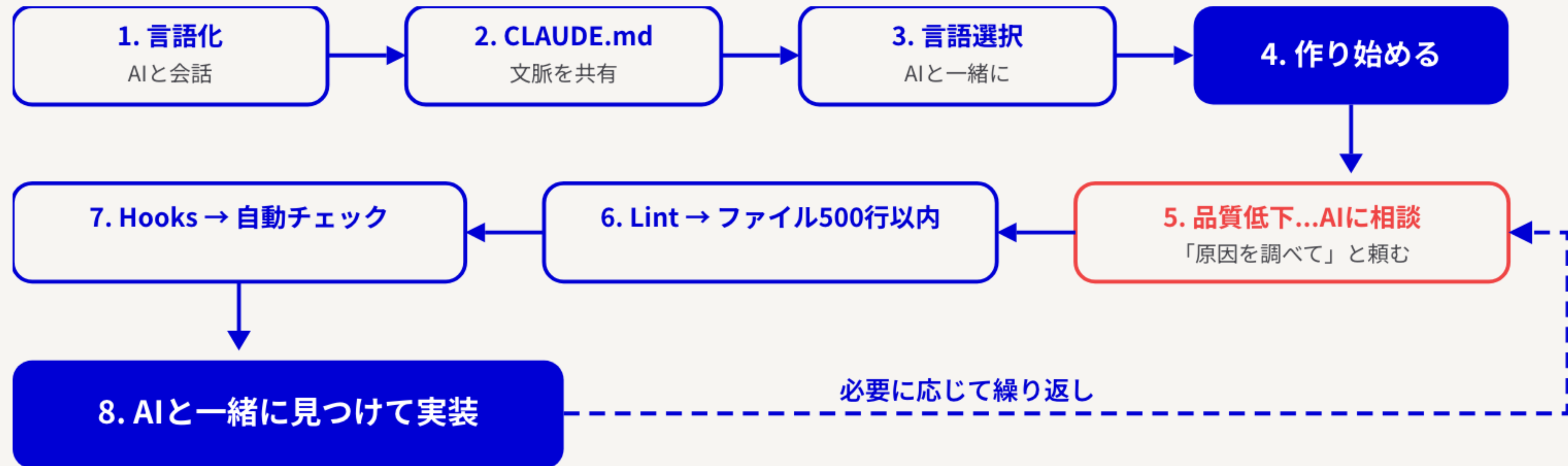
厳格に設定すると、AIが常に綺麗なコードを書くようになる。

AI駆動開発

AI-Driven Development



「学んでからやる」ではなく「やりながら学ぶ」



AIと喋りながら、作りながら学んでいく。

配布内容

テンプレートに含まれるもの

```
terackathon-template/  
├── .claude/  
│   ├── settings.json      ← セキュリティ設定  
│   └── hooks/  
│       └── security-guard.sh ← セキュリティガード  
│   ├── skills/            ← スキル（ドキュメント管理等）  
│   └── agents/             ← サブエージェント（5種類）  
├── CLAUDE.md              ← AIへの指示書  
├── .env.example            ← APIキーテンプレート  
└── .gitignore              ← .env除外
```

テンプレートの構造

クローンするだけで全設定が適用される。設定ファイルを自分で書く必要なし。

入手方法

```
git clone https://github.com/ai-ops-manager/terackathon-template
```

※ URLはこの場で共有します



CLAUDE.md

配布したCLAUDE.mdの内容

今回配布するCLAUDE.mdには、初心者がAI駆動開発を始めるためのルールが書かれている。

AIの進め方（askベース）

- まず聞く — 何を作りたいかを一緒に言語化
- 設計する — 技術選択・構造を提案して承認を得る
- 調べる — 必要な技術を説明しながら進める
- 作る — 小さく作って動作確認しながら
- 説明する — 何をしたか必ず日本語で説明

サブエージェントの活用

メインのAIはPMとして全体を管理。実作業はサブエージェント（builder, researcher, reviewer, tester, documenter）に委任して並列処理する。

settings.json

配布したsettings.jsonが制御していること

AIが実行できるコマンド・読めるファイルを制限している。

許可しているもの

- npm install / yarn add (パッケージ追加)
- git clone (リポジトリのコピー)

禁止しているもの

- .envファイルの読み取り
- rm -rf / sudo / chmod (危険なコマンド)
- curl / wget / ssh (ネットワークツール)
- git push --force (強制プッシュ)
- SSH鍵・AWS認証情報へのアクセス

Hooks

security-guard.shが守っていること

settings.jsonの制限をすり抜ける攻撃を、シェルスクリプトで確実にブロックする。

検知する攻撃パターン

- シークレットファイルへの間接アクセス（cat, head, tail等で.envを読む）
- シンボリックリンク経由の迂回攻撃
- 環境変数のダンプ（APIキーが含まれる）
- インラインスクリプトによるバイパス
- クラウド認証ファイルへのアクセス
- セキュリティ設定ファイル自体の書き換え

settings.json = リクエスト（バイパスされる可能性あり）

Hooks = 法律（確実にブロック。バイパス不可能）

AIへの指示

困ったらそのまま伝える

エラーメッセージをそのまま貼るだけでOK。具体的であるほど良い結果が出る。

良い指示の例

- 「このプロジェクトの構成を教えて」
- 「Hello Worldのアプリを作って」
- 「〇〇がエラーで動かない」 + エラーメッセージ貼り付け
- 「この関数を修正して、テストも書いて」
- 「最近コードの質が落ちてる。原因を調べて」

ポイント

- 1回の指示で1つのことを頼む
- 変更提案は必ず確認してから承認
- わからないことはAIに聞く

まとめ

1. ガードレールを先に敷く

settings.json + Hooks + Lintで自動化する。

2. やりながら学ぶ

AIと会話しながら、必要なものをその場で見つけて実装する。

3. 困ったらAIに聞く

エラーもわからないことも、そのまま伝えればいい。



テラッカソン Claude Code研修

ご質問はいつでもどうぞ

